Laboratory Work III: Acoustic FWI

Full Waveform Inversion using Adjoint-State Method

Group Members:
Shahariar Ryehan
Marti Arjona
Pranav Prakasan

M.Sc. in Acoustic and Vibration Engineering

Contents

1 Introduction

In seismic imaging, the primary goal is to reconstruct the physical properties of the subsurface, such as wave velocity and density, using wavefields measured at the surface. This is a nonlinear inverse problem because the relationship between the model parameters m and the observed data d is governed by the wave equation.

Full Waveform Inversion (FWI) exploits the complete wavefield information, including phase and amplitude, to iteratively recover medium parameters by minimizing a misfit function:

$$J(m) = \frac{1}{2}\|d_{obs} - d_{calc}(m)\|^2 \tag{1.a}$$

where d_{obs} denotes the observed data and $d_{calc}(m)$ the modeled data.

2 Part 1: Understanding the Forward Problem

2.1 Exercise 1.1: Forward Modeling

The velocity model was modified by $\Delta c = \pm 10\%$ and synthetic data were generated in both time and frequency domains.

a) Do you observe differences in arrival times? Why?

Yes. Since travel time is inversely proportional to velocity, increasing velocity leads to earlier arrivals, while decreasing velocity delays arrivals.

b) How are the amplitudes and phase affected?

Velocity changes modify impedance contrasts, affecting amplitudes through reflection and transmission. Phase shifts occur because wave speed controls the temporal evolution of the wavefield.

c) Signal Analysis

The signal was approximated as $f(t) \approx A\cos(\omega t + \phi)$ using the following MATLAB code:

```
%% Exercise 1: Forward Modeling
clear all; close all; clc;

% 1. Setup Domain (Same as your main script)
fmax = 10;
nx = ceil(10*fmax);
dom = domain([0 1 0 1], [nx nx]);

% 2. Setup Velocity Model (Perturbed Model for Ex 1.1)
c_background = 1 * ones(nx^2, 1);       % Uniform background
c_perturbed  = c_background * 1.1;      % +10% perturbation
m_background = 1./c_background.^2;      % Convert to squared slowness
m_perturbed  = 1./c_perturbed.^2;

% 3. Define a single source in the middle
source_loc = [0.5; 0.5];
b = generate_sources(dom, source_loc);

% 4. Solve Forward Problem at specific Frequency
freq = 5; % Change this for Ex 1.2 (e.g., 3Hz, 5Hz, 10Hz)

% Solve for Background
fprintf('Solving for Background...\n');
```

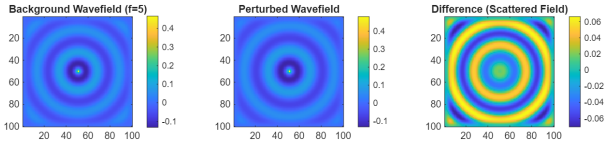

Figure 1: Forward Modeling

```
H_back = helmholtz_2d(m_background, freq, dom);
A_back = invertA(H_back, 1);
u_back = A_back.apply(b);

% Solve for Perturbed
fprintf('Solving for Perturbed...\n');
H_pert = helmholtz_2d(m_perturbed, freq, dom);
A_pert = invertA(H_pert, 1);
u_pert = A_pert.apply(b);

% 5. Visualize Wavefields (Real part)
figure;
subplot(1,3,1);
dom.imagesc(real(u_back)); title(['Background Wavefield (f=' num2str(
    freq) ')']);
axis square; colorbar;

subplot(1,3,2);
dom.imagesc(real(u_pert)); title('Perturbed Wavefield');
axis square; colorbar;

subplot(1,3,3);
dom.imagesc(real(u_pert - u_back)); title('Difference (Scattered Field)
    ');
axis square; colorbar;
%% 6. Visualize 1D Cross-section (Slice) to see Phase Shift
% Convert the vector u to a 2D matrix
U_back_2D = reshape(real(u_back), nx, nx);
U_pert_2D = reshape(real(u_pert), nx, nx);

% Select the middle row index
mid_row = ceil(nx/2);

% Extract the slice
slice_back = U_back_2D(:, mid_row); % Slicing through the middle
slice_pert = U_pert_2D(:, mid_row);

% Plot comparison
figure('Name', '1D Cross-Section Comparison');
plot(slice_back, 'b', 'LineWidth', 1.5); hold on;
plot(slice_pert, 'r--', 'LineWidth', 1.5);
legend('Background (c=1)', 'Perturbed (c=1.1)');
xlabel('Grid Points (x)');
ylabel('Amplitude (Real part of p)');
title(['Wavefield Slice at f = ' num2str(freq) ' Hz']);
grid on;
```

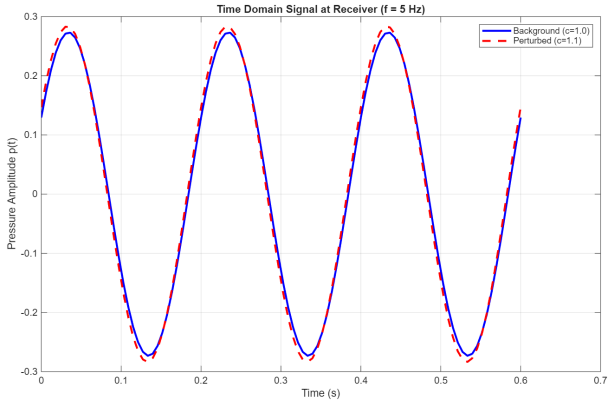

Figure 2: 10parcent

2.2 Exercise 1.2: Frequency Sensitivity Analysis

a) Relationship between frequency and wavelength

The wavelength is given by $\lambda = c/f$. Higher frequencies correspond to shorter wavelengths.

b) Influence on spatial resolution

Shorter wavelengths improve resolution of small anomalies but increase sensitivity to noise and cycle skipping. Lower frequencies provide stability at the cost of resolution.

3 Part 2: Gradients and Sensitivity Kernels

3.1 Exercise 2.1: Sensitivity Kernels

Sensitivity kernels exhibit a characteristic banana-doughnut shape, with maximum sensitivity in the Fresnel zone surrounding the ray path.

b) Effect of acquisition geometry

Dense and wide-angle source–receiver coverage improves illumination and resolution of the subsurface.

3.2 Exercise 2.2: Gradient Computation

The adjoint-state method computes gradients efficiently by correlating forward and adjoint wavefields.

a) Where is the gradient strongest?

Near sources and receivers, where wavefield energy is highest.

b) Comparison with true anomaly

The gradient indicates the descent direction and does not perfectly match the anomaly in early iterations.

```
    %% Exercise 2: Sensitivity Kernels
clear all; close all; clc;

% Setup
fmax = 10;
nx = ceil(10*fmax);
dom = domain([0 1 0 1], [nx nx]);
c0 = ones(nx^2, 1);
```

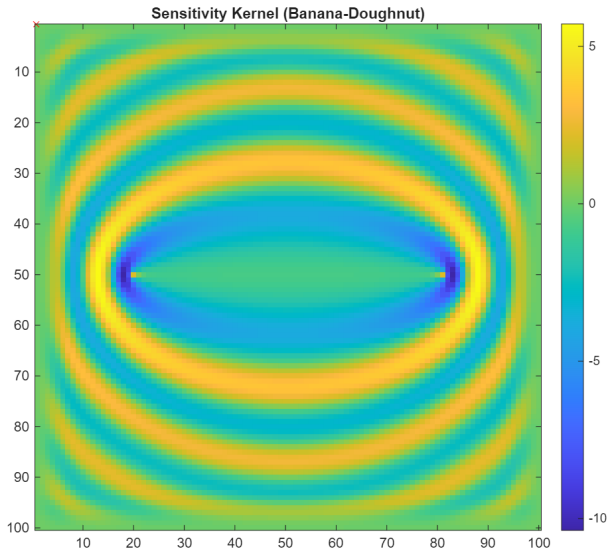

Figure 3: banana

```
m0 = 1./c0.^2;
freq = 5; % Chosen frequency

% Define 1 Source and 1 Receiver
src_loc = [0.2; 0.5]; % Left side
rec_loc = [0.8; 0.5]; % Right side

% Generate Source Vector (b) and Receiver Vector (bq)
b = generate_sources(dom, src_loc);
bq = generate_sources(dom, rec_loc); % Receiver acts as a source for
    adjoint

% Solve Forward Wavefield (u)
fprintf('Solving␣Forward...\n');
H = helmholtz_2d(m0, freq, dom);
A = invertA(H, 1);
u = A.apply(b);

% Solve Adjoint Wavefield (q) - Backpropagate from receiver
fprintf('Solving␣Adjoint...\n');
q = A.applyt(bq); % applyt is transpose (adjoint)

% Calculate Sensitivity Kernel: K = w^2 * u * q
omega = 2 * pi * freq;
K = omega^2 * real(u .* conj(q)); % Eq (7) in manual

% Plot
figure;
dom.imagesc(K);
hold on; plot(src_loc(1), src_loc(2), 'go'); plot(rec_loc(1), rec_loc
    (2), 'rx');
title('Sensitivity␣Kernel␣(Banana-Doughnut)');
axis square; colorbar;
```

4 Part 3: Practical Inversion

4.1 Exercise 3.1: Inversion with Perfect Synthetic Data

Using a Shepp–Logan phantom, the inversion successfully recovered major structures.

b) Cost function evolution

The objective function decreases monotonically, indicating convergence.

c) Artifacts

Artifacts appear near boundaries and source locations due to limited aperture and bandwidth.

```
    function [m,out]=adjoint_state_2d(dom,all_freqs,sources,receivers,
       window_info,c_true,c_0,sigma,maxit)
%ADJOINT_STATE_2D   Adjoint-state method for full-waveform inversion in
    2D.
%   [M,OUT] = ADJOINT_STATE_2D(DOM,ALL_FREQS,SOURCES,RECEIVERS,
   WINDOW_INFO,C_TRUE,C_0,SIGMA,MAXIT)
%   runs the adjoint-state implementation of full-waveform inversion.
   The
%   domain object DOM describes the physical domaind and grid used. The
%   vector ALL_FREQS is a list of all desired frequencies in hertz. The
    two
%   row matrices SOURCES and RECEIVERS contain x and y locations of
   sources
%   and receivers in the physical domain, and should be generated using
%   sources_and_receivers. Window_info is a struct containing
   information
%   about windowing; see dom.window for more. C_TRUE and C_0 are the
   true
%   and initial background velocities. SIGMA is the noise level and
   MAXIT
%   is the maximum number of permitted iterations.
%
%       Returned are a variable M, sometimes called the "squared
   slowness", and
%       a struct out with information coming directly from the LBFGS
   routine.

N = dom.N;
                    % total number of interior grid points
[win_inds,~,W] = dom.window(window_info);       % window indices and
   prolongation from window to domain
nf = length(all_freqs);                                          %
   number of frequencies
ns = size(sources,2);                                            %
   number of sources
nw = length(win_inds);                                           %
   number of pixels outside window
r_ind = dom.loc2ind(receivers);                          % flat indices
   of receivers
I = speye(N);
           % for padding/prolongation operators
E_rec = I(:,r_ind);
           % padding operator
m_true = 1./c_true.^2;                                           % true
   sqaured slowness
m0 = m_true;
```

```
            % initial squared slowness, true value inside window
m0(win_inds) = 1./c_0(win_inds).^2;                    % set values
    inside of window to initial value
b = generate_sources(dom,sources);                     % vectors
    corresponding to source locations
opt.Niter = maxit;
            % setup opt for LBFGS

% Measure noisy data of true fields at receiver locations
d = generate_seismic_data(dom,sources,receivers,all_freqs,m_true,sigma)
    ;

A3=abs(d)
phase = atan((imag(d))./(real(d)))
t = linspace(1,5,100)

f=A3(2,5,6).*cos(3*t+phase(2,5,6))
plot(t,f)
keyboard
% Run LBFGS with no initial perturbation
[dm,out] = lbfgs(@adjoint_state_gradient,zeros(nw,1),opt);

% Return updated m
m = m0+W*dm;

    function [J,DJ] = adjoint_state_gradient(dm)
        % Compute the gradient of J(m) = .5*|| S*u(m) - d ||^2 via the
            adjoint state method
        fprintf('Solving Helmholtz, frequency: ')
        DJ = zeros(nw,1);       % gradient
        J = 0;                          % value of objective function
        % Parallelize over frequencies
        for ii = 1:nf
            fprintf('%d, ',ii)
            A = invertA(helmholtz_2d(m0+W*dm,all_freqs(ii),dom),1);
            for jj = 1:ns
                u = A.apply(b(:,jj)); %#ok<PFBNS>       % Forward solve
                bq = E_rec*(u(r_ind)-d(:,ii,jj));       % Padding
                q = A.applyt(bq);
                            % Adjoint solve
                % kernel
                % kk = real(2*pi*all_freqs(ii))^2*real(u(win_inds).*
                    conj(q(win_inds)));
                % K = zeros(1,100*100);
                % K(win_inds) = K(win_inds) + kk';
                % figure
                % surf(1:100,1:100,reshape(K,100,100))
                % view(2)
                % Update the gradient
                DJ = DJ+(2*pi*all_freqs(ii))^2*real(u(win_inds).*conj(q
                    (win_inds)));
                % plot(DJ);
                % Update the objective value
                J = J+norm(u(r_ind)-d(:,ii,jj))^2;
            end
        end
        JJ = zeros(1,100*100);
        JJ(win_inds) = JJ(win_inds) + DJ';
```

```
76          surf(1:100,1:100,reshape(JJ,100,100))
77          view(2)
78          fprintf('\n')
79      end
80  end
```

4.2 Exercise 3.2: Multi-Scale Strategy

A progressive frequency strategy provides the best reconstruction by avoiding cycle skipping.

4.3 Exercise 3.3: Noise Robustness

Increasing noise degrades high-frequency information first.

Regularization

Tikhonov regularization and gradient smoothing improve robustness.

5 Part 4: Bonus Questions

5.1 Acquisition Geometry

Wide angular coverage yields better resolution and reduces smearing.

5.2 Influence of Initial Model

Convergence occurs only when the initial model lies within the basin of attraction of the true solution.